



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Análisis y Desarrollo de Software (ADSO).
- **Código del Programa de Formación:** 228118
- **Nombre del Proyecto Formativo:** Construcción de software a la medida con arquitectura cloud, interfaces modernas y desarrollo guiado por inteligencia artificial para el sector productivo empresarial.
- **Fase del Proyecto:** Ejecución
- **Actividad de Proyecto Formativo:** AP03 — CODIFICAR Y DESPLEGAR LOS MÓDULOS DEL SISTEMA DE INFORMACIÓN APLICANDO METODOLOGÍAS DE DESARROLLO GUIADO POR INTELIGENCIA ARTIFICIAL.
- **Competencia:** 220501096 — Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo.
- **Resultados de Aprendizaje a Alcanzar:**
 - **220501096-01:** Modelar la arquitectura y requisitos del software aplicando metodologías BDD y la Tetralogía Documental Canónica.
 - **220501096-02:** Construir la base de datos relacional y las políticas de seguridad por fila (RLS) en Supabase PostgreSQL como Fuente Única de Verdad (SSoT).
 - **220501096-03:** Diseñar e iterar prototipos de interfaz de usuario en Google Stitch incorporando el catálogo de estilos y los 4 estados obligatorios de pantalla.
 - **220501096-04:** Codificar e integrar aplicaciones fullstack reactivas en Google AI Studio Apps garantizando persistencia en tiempo real y arquitectura modular.
- **Duración de la Guía de Aprendizaje:** 120 horas (Trabajo directo con instructor: 36 horas; Trabajo autónomo y colaborativo en proyecto: 84 horas).

2. PRESENTACIÓN

- Motivar hacia la actividad de aprendizaje en consideración a las fortalezas que aportará en el desarrollo de habilidades y destrezas.
- Guiar y organizar el aprendizaje de manera que se oriente al desarrollo integral del aprendiz.
- Motivar a la acción, al trabajo autónomo sistemático y organizado.
- Relacionar conocimientos previos con los nuevos para la construcción significativa de los mismos.
- Promover el aprendizaje colaborativo y el crecimiento integral del grupo.

Estimado aprendiz, la industria global del software atraviesa la transformación más profunda de las últimas décadas. La emergencia de modelos fundacionales de Inteligencia Artificial Generativa ha redefinido el rol del programador: ya no somos meros transcritores manuales de sintaxis, sino **arquitectos de sistemas, directores de contexto y garantes de la seguridad y calidad del software**.

Sin embargo, el uso indiscriminado de la inteligencia artificial sin un marco metodológico riguroso suele conducir al fracaso conocido como *"Vibe Coding"* o desarrollo por improvisación. Cuando un desarrollador solicita código a una IA sin delimitar el alcance, sin un esquema relacional formal y sin especificar los estados de interfaz, el sistema inevitablemente alucina: inventa datos simulados (*mocks* en memoria), genera componentes monolíticos inmantenibles (*God Files*), expone credenciales secretas y degrada su coherencia conceptual (*Understanding Rot* y *Context Rot*).

Para formar profesionales con estándares de producción de nivel internacional, esta guía adopta el **Marco PIC 2026 (Precision, Isolation & Context Framework)** y la **Tetralogía Documental Canónica (Plan, PRD, User Flow y TRD)**, articulando un pipeline tecnológico de cuatro plataformas sinérgicas:

- **Google Gemini (Gemini Especializadas):** Mentor socrático para ingeniería de requisitos, modelado relacional y especificaciones técnicas sin alucinaciones.
- **Google Stitch (stitch.withgoogle.com):** Prototipado visual con tokens de diseño y especificación contractual de los 4 estados de UI.
- **Supabase PostgreSQL:** Backend como Servicio (BaaS) que actúa como *Fuente Única de Verdad (SSoT)* con RLS al 100%, triggers y UUIDs.
- **Google AI Studio (aistudio.google.com/apps):** Compilación fullstack reactiva que traduce los planos en código modular conectado en tiempo real.

A través de esta guía, usted construirá paso a paso una aplicación web completa y profesional para su Proyecto Formativo, erradicando la improvisación y dominando las competencias de ingeniería de software más demandadas en el sector productivo. ¡Bienvenido a la era del Desarrollo Guiado por IA!



3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

• Descripción de la(s) Actividad(es)

3.1 Actividades de reflexión inicial:

Descripción de la actividad: Análisis reflexivo individual y grupal sobre el impacto de construir software con IA guiada por planos formales frente a la improvisación no asistida. Identificación de anti-patrones en el desarrollo con herramientas generativas.

Ambiente requerido: Ambiente de formación de informática con conexión a Internet y proyector / pantalla interactiva.

Estrategias o técnicas didácticas activas: Estudio de caso problematizador, Aprendizaje Basado en Problemas (ABP), lluvia de ideas y debate socrático guiado.

Materiales de formación: Estaciones de trabajo, navegador web moderno, libreta técnica de notas.

Material de apoyo: Documento de caso de estudio: "El Espejismo del Prompt Mágico: Anatomía de un SaaS Fallido".

Duración de la actividad: 6 horas.

3.1.1 Estudio de Caso: "El Espejismo del Prompt Mágico"

Situación Problematicadora: Un desarrollador junior solicita a un modelo de IA generativa comercial: <i>"Crea una aplicación web moderna para citas médicas con login, doctores y pagos"</i>. El modelo entrega en segundos una pantalla visualmente atractiva en React. Sin embargo, al realizar pruebas de usuario surgen los siguientes fallos críticos: - Los datos de citas se almacenan en un arreglo temporal con <code>useState([])</code>. Al recargar la página (F5), todos los datos desaparecen. - Cualquier usuario puede ver y modificar las historias clínicas de otros pacientes al carecer de políticas de seguridad en base de datos (RLS inexistente). - Al solicitar un ajuste menor en la interfaz, la IA reescribe todo el archivo generando errores de sintaxis y eliminando funcionalidades previamente operativas.	Anti-patrón: Simulación en Memoria vs SSoT <pre>// CÓDIGO DEFECTUOSO GENERADO POR IMPROVISACIÓN: const [citas, setCitas] = useState([{ id: 1, doctor: "Dr. Mock", fecha: "Hoy" }, { id: 2, doctor: "Dra. Fake", fecha: "Mañana" }]); // ALUCINACIÓN: Datos hardcodeados no persistentes const agendarCita = (nuevaCita) => { // ERROR CRÍTICO: No persiste en PostgreSQL setCitas([...citas, nuevaCita]); alert("Cita creada con éxito"); }; // CONSECUENCIA: Pérdida total de datos en recarga y // violación flagrante de privacidad entre usuarios.</pre> Fallo Estructural: Ausencia de Fuente Única de Verdad (SSoT), falta de arquitectura relacional y omisión de especificación de requisitos BDD.
---	--

3.1.2 Preguntas Orientadoras para la Socialización

De forma individual, responda las siguientes interrogantes en su bitácora técnica para posteriormente debatirlas en plenaria grupal:

- ¿Por qué los modelos de lenguaje tienden a generar datos simulados (`useState` , `setTimeout`) cuando no se les suministra un esquema SQL formal?
- ¿Qué implicaciones legales y éticas tendría desplegar en una clínica real una aplicación que carece de Row Level Security (RLS) en su motor de persistencia?
- Si un arquitecto tradicional no inicia una edificación sin planos estructurales, ¿por qué muchos programadores intentan codificar software solicitándole código a la IA sin un Plan, un PRD, un User Flow y un TRD?

Conclusión Pedagógica: La Inteligencia Artificial no reemplaza la disciplina de la ingeniería; por el contrario, amplifica exponencialmente las decisiones arquitectónicas del desarrollador. Una buena especificación documental produce código de nivel empresarial; la ausencia de especificación produce alucinaciones costosas.



3.2 Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje:

- Descripción de la actividad:** Apropiación conceptual de los fundamentos del Marco PIC 2026, la Tetralogía Documental Canónica, el principio de SSoT en bases de datos relacionales con RLS y el diseño reactivo de 4 estados de pantalla.
- Ambiente requerido:** Ambiente de desarrollo de software con acceso a plataformas web.
- Estrategias o técnicas didácticas activas:** Exposición dialogada del instructor, organizadores gráficos conceptuales, análisis comparativo de matrices y lectura crítica guiada.
- Materiales de formación:** Equipos con conexión a Internet, diagramas de arquitectura en formato digital.
- Material de apoyo:** Documentos técnicos `README.md` , `AGENTS.md` y `CADENA_DE_GEMAS_Y_CONOCIMIENTOS.md` .
- Duración de la actividad:** 14 horas.

3.2.1 Matriz Comparativa de Paradigmas de Desarrollo

Criterio	Desarrollo Tradicional Manual	Improvisación ("Vibe Coding")	Desarrollo Guiado por IA (Marco PIC 2026)
Punto de Partida	Requisitos estáticos en cascada o historias de usuario en tableros.	Prompts abiertos y vagos en interfaces de chat estándar.	Tetralogía Documental Canónica (Plan, PRD, User Flow, TRD).
Velocidad de Iteración	Lenta; escritura manual de código boilerplate y configuración.	Aparentemente rápida al inicio, colapsa en la integración.	Óptima y predecible ; código modular compilado en minutos con IA.
Fuente de Verdad (SSoT)	Servidor backend / ORM tradicional manual.	Inexistente o ambigua; abundancia de datos mockeados.	Supabase PostgreSQL : RLS al 100%, triggers y UUIDs nativos.
Consistencia Visual UX	Depende de bibliotecas CSS o maquetadores manuales.	Interfaces incoherentes sin estados de error ni carga.	Google Stitch : Tokens visuales y 4 estados de pantalla obligatorios.
Tolerancia a Errores	Alta depuración manual mediante breakpoints y logs.	Degradación continua (<i>Context Drift</i>) al corregir bugs.	Protocolo de Parches Quirúrgicos y verificación <i>Maker-Checker</i> .

3.2.2 El Pipeline de los 7 Artefactos Canónicos

El estándar de desarrollo guiado establece que antes de solicitar código ejecutable a un modelo de lenguaje, se deben compilar estrictamente los 7 artefactos de la cadena:

- [01_PLAN] Plan de Proyecto:** Alcance del MVP (In-Scope vs Out-of-Scope), dependencias y sprints.
- [02_PRD] PRD:** Personas, Requerimientos Funcionales (RF) con sintaxis Gherkin BDD y Reglas de Negocio.
- [03_FLOW] User Flow UX:** Diagramas de navegación y los 4 estados de interfaz.
- [04_TRD] TRD:** Stack tecnológico exacto, modelo ERD y seguridad.
- [05_SQL] Esquema Supabase:** DDL relacional, RLS y triggers.
- [06_STITCH] Prompts Google Stitch:** Estilos y maquetación UI.
- [07_AI_STUDIO] Prompt Maestro:** Orquestación fullstack.

Flujo Encadenado de Artefactos

```
graph TD; A["1. GEMA ARQUITECTO SDLC (Gemini)  
→ Genera: 01_PLAN + 02_PRD + 03_FLOW + 04_TRD"] --> B["2. GEMA DBA SUPABASE  
Alimenta: 04_TRD  
→ 05_ESQUEMA.sql"]; A --> C["3. GEMA STITCH UI/UX  
Alimenta: 02 y 03  
→ 06_STITCH.md"]; B --> D["4. GEMA ORQUESTADOR FULLSTACK (AI Studio)  
Alimenta: 04_TRD + 05_SQL + 06_STITCH  
→ Genera: 07_PROMPT_MAESTRO_AISTUDIO.md  
→ Compila: SPA React 19 + Supabase en Vivo"]; C --> D;
```

Regla de Oro del Marco PIC: Cada documento de la Tetralogía resuelve una única dimensión del problema (Aislamiento de Contexto). Al ingresarlos en la memoria de "Conocimientos" de Gemini o AI Studio, el modelo opera con precisión quirúrgica y cero alucinaciones.



3.3 Actividades de apropiación:

- Descripción de la actividad:** Ejecución práctica y guiada de las fases del pipeline: creación de la cadena de Gemas en Gemini, elaboración de la Tetralogía Canónica, prototipado en Google Stitch, modelado DDL con RLS en Supabase y orquestación fullstack en Google AI Studio Apps.
- Ambiente requerido:** Laboratorio de desarrollo con cuentas habilitadas en Google Gemini, Google AI Studio, Google Stitch y Supabase Cloud.
- Estrategias o técnicas didácticas activas:** Taller práctico guiado paso a paso, codificación estructurada, demostración técnica y pair-programming.
- Materiales de formación:** Navegador web, cliente SQL / Dashboard Supabase, editor de texto o IDE.
- Evidencias de aprendizaje:** Artefactos documentales generados, script SQL ejecutado con RLS verificado y aplicación web reactiva funcional.
- Instrumentos de evaluación:** Lista de verificación de producto y rúbrica técnica de desempeño.
- Duración de la actividad:** 60 horas.

3.3.1 Configuración de la Cadena de 4 Gemas Especializadas en Gemini

Paso 1: Acceso a Gemini Gems: Ingrese a <code>gemini.google.com</code>, despliegue el menú lateral izquierdo y seleccione "Explorar Gemas" → "Nueva Gema". Configuración de la GEMA 1: • Nombre: <code>01 – Arquitecto de Producto & SDLC</code> • Descripción: Transforma ideas iniciales en los 4 documentos canónicos (Plan, PRD, User Flow, TRD) mediante diálogo socrático riguroso. • Conocimientos (Knowledge): Vacío al inicio (es el punto de origen del pipeline). Copie el System Prompt maestro adjunto en el campo de instrucciones de la Gema.	System Prompt Maestro: Gema 1 (Arquitecto) <pre># ROL Y MISIÓN Eres un Lead Software Architect y Senior Product Manager. Tu misión es guiar al aprendiz paso a paso mediante diálogo socrático para generar la Tetralogía Documental Canónica. NUNCA inventes requerimientos: pregunta siempre al usuario. # REGLAS DE TRABAJO EN 4 PASOS 1. PLAN: Delimita MVP In-Scope vs Out-of-Scope y Sprints. -> Genera: 01_PLAN_PROYECTO.md 2. PRD: Perfiles y Requerimientos Funcionales en Gherkin. -> Genera: 02_PRD_PRODUCTO.md 3. USER FLOW: Diagrama Mermaid y 4 estados de pantalla (Empty, Loading Skeleton, Success Toast, Error Alert). -> Genera: 03_USER_FLOWS_UX.md 4. TRD: Stack tecnológico exacto, modelo ERD y RLS. -> Genera: 04_TRD_ARQUITECTURA_TECNICA.md # FORMATO DE SALIDA: Bloques Markdown descargables.</pre>
Paso 2: Cadena y Distribución de Conocimientos: A medida que la GEMA 1 finaliza los documentos, el aprendiz descarga los archivos <code>.md</code> y los añade en la sección "Conocimientos" (Knowledge) de las gemas hijas especializadas: • GEMA 2 (DBA Supabase): Se alimenta de <code>04_TRD_ARQUITECTURA_TECNICA.md</code> para generar el script SQL DDL con RLS al 100%. • GEMA 3 (Diseñador Stitch UI): Se alimenta de <code>02_PRD</code> y <code>03_USER_FLOWS</code> para producir los prompts visuales. • GEMA 4 (Orquestador AI Studio): Recibe <code>04_TRD</code>, <code>05_SQL</code> y <code>06_STITCH</code> para ensamblar el prompt maestro fullstack.	Panel de Conocimientos en Google Gemini 📁 Archivos en Conocimientos de la Gema: ✓ <code>01_PLAN_PROYECTO.md</code> (14 KB) ✓ <code>02_PRD_PRODUCTO.md</code> (22 KB) ✓ <code>03_USER_FLOWS_UX.md</code> (18 KB) ✓ <code>04_TRD_ARQUITECTURA_TECNICA.md</code> (25 KB) <i>"Los archivos adjuntos proporcionan contexto permanente y evitan el olvido conceptual durante largas sesiones."</i>

Tip de Productividad: No intente generar todo el sistema en una sola conversación de chat libre. La división del trabajo en Gemas especializadas previene la saturación del contexto de la IA y garantiza código libre de alucinaciones.



3.3.2 Elaboración Paso a Paso de la Tetralogía Documental Canónica

Documento 1: Plan de Proyecto (01_PLAN_PROYECTO.md) Es la brújula del desarrollo. Delimita el alcance mínimo viable para que la IA no intente construir funcionalidades accesorias innecesarias. Estructura Obligatoria: • Visión del MVP: Propósito central en una frase. • Límites de Alcance (Scope Boundaries): ◦ [IN-SCOPE] Autenticación Supabase (Email/Password), CRUD de entidad principal, Dashboard con KPIs en tiempo real y UI en 4 estados. ◦ [OUT-OF-SCOPE] Pagos internacionales (Stripe), notificaciones push móviles, analítica con ML predictivo (pospuestos para v2.0). • Matriz de Dependencias: DDL SQL y RLS → Auth → Prototipos Stitch → Compilación AI Studio.	Fragmento del Plan de Proyecto <pre># 📄 PLAN DE PROYECTO: HEALTHPULSE TELEMED ## 1. VISIÓN DEL MVP Plataforma SaaS para que médicos independientes gestionen pacientes, citas y notas clínicas seguras. ## 2. LÍMITES DEL ALCANCE - IN-SCOPE (MVP Día 1-3): * Autenticación con roles (doctor, patient). * CRUD de Citas (Appointments) vinculadas a user.id. * Dashboard con 4 KPIs en tiempo real. * UI con Skeleton Screens y Toast Notifications. - OUT-OF-SCOPE (Versión 2.0): * Pasarela de pagos con tarjeta de crédito. * Salas de videoconferencia WebRTC nativas.</pre>
Documento 2: PRD con Sintaxis Gherkin BDD (02_PRD_PRODUCTO.md) El PRD documenta el comportamiento esperado del sistema mediante el lenguaje formal Gherkin (Dado / Cuando / Entonces), eliminando cualquier ambigüedad de interpretación para la IA. Cada Requerimiento Funcional (RF) debe contar con: • Código y Nombre: Ej. RF-02: Agendamiento de Citas • Prioridad MoSCoW: Must Have / Should Have. • Actor: Rol autorizado para ejecutar la acción. • Criterio de Aceptación Gherkin: Escenario de éxito y escenario de excepción o validación de error.	Especificación BDD en Gherkin <pre>#### RF-02: Agendamiento de Cita Médica - Prioridad: Must Have Actor: Paciente autenticado Escenario: Paciente agenda cita con éxito DADO que el paciente ha iniciado sesión Y selecciona al médico "Dr. Carlos Mendoza" Y elige la fecha "2026-10-12" a las "10:00 AM" CUANDO hace clic en el botón "Confirmar Cita" ENTONCES el sistema inserta la fila en 'appointments' Y el campo 'owner_id' toma el valor de auth.uid() Y la interfaz emite un Toast verde "Cita agendada" Y la tabla de citas se actualiza reactivamente. Escenario: Conflicto de horario no disponible DADO que el horario seleccionado ya se encuentra ocupado CUANDO el paciente intenta seleccionarlo ENTONCES el bloque se muestra deshabilitado en gris Y el sistema bloquea el botón de confirmación.</pre>



Documento 3: User Flows, Catálogo de Pantallas & 4 Estados de UI (03_USER_FLOWS_UX.md) El User Flow traza la navegación contractual del sistema. Para erradicar el "Efecto Monovista", el aprendiz debe definir obligatoriamente un Catálogo de Pantallas (Screen Inventory) con mínimo 5 a 8 vistas independientes: • SCR-01: Auth & Onboarding (Login / Registro / Recuperación). • SCR-02: Dashboard Principal (4 KPIs en vivo + resumen). • SCR-03: Explorador de Registros (Tabla + selector Kanban). • SCR-04: Detalle 360 del Registro (Ficha técnica profunda). • SCR-05: Formulario de Creación (Wizard modular). • SCR-06: Configuración & Perfil de Usuario. Para cada pantalla se especifican contractualmente sus 4 estados: 1. Empty State: Consulta retorna 0 filas. Ilustración amigable + CTA. 2. Skeleton Screen: Réplica pulsante animada mientras carga Supabase. 3. Toast Feedback: Notificación flotante accesible de confirmación. 4. Error & Retry: Manejo de excepciones (red/RLS) con botón reintentar.	Navegación Global Multi-Vista (Mermaid) <pre>graph TD A((● Visitante)) --> B{¿Sesión activa?} B -- No --> C[SCR-01: Auth Login/Registro] B -- Sí --> D[SCR-02: Dashboard 4 KPIs] C --> D D --> E[Login Exitoso] subgraph APP[SPA MULTI-VISTA - SIDEBAR NAVEGABLE] D <--> Nav[Registros] Nav --> E[SCR-03: Explorador / Tabla] D <--> H[Ajustes] H --> F[SCR-06: Perfil & Config] E --> G[Clic Fila / Ver Ficha] G --> F[SCR-04: Detalle 360] F --> I[Volver] I --> E F --> J[Botón '+ Nuevo'] J --> K[SCR-05: Formulario Wizard] K --> L[Guardado Exitoso] L --> E end</pre>
Documento 4: TRD y Arquitectura Técnica (04_TRD_ARQUITECTURA_TECNICA.md) El TRD fija los contratos técnicos invariantes para evitar que la IA utilice versiones obsoletas o paquetes incompatibles: • Stack Frontend: React 19 SPA, Tailwind CSS v3.4+, Lucide React Icons, cliente <code>@supabase/supabase-js</code> v2.x. • Arquitectura de Directorios: Modular basada en características (Feature-Driven), separando servicios (<code>services/</code>), contexto (<code>context/</code>), componentes UI base (<code>components/ui/</code>) y componentes de dominio (<code>components/features/</code>). • Límite de Contexto: Ningún componente puede superar las 250 líneas de código para evitar <i>Context Rot</i>.	Estructura Modular Estandarizada <pre>src/ ├── lib/ │ └── supabaseClient.js # Singleton con persistencia ├── context/ │ └── AuthContext.jsx # Provider y hook useAuth() ├── services/ │ ├── profileService.js # Consultas a public.profiles │ └── appointmentService.js # CRUD sobre appointments ├── components/ │ ├── layout/ # Sidebar, Header, KPI Cards │ ├── ui/ # Skeletons, Toasts, Modales │ └── features/ # Tablas de datos, Formularios ├── types/ # Definición de tipos y esquema └── App.jsx # Enrutamiento condicional</pre>

Tip de Arquitectura: Al modularizar el código en archivos de menos de 250 líneas, la IA puede realizar modificaciones incrementales mediante parches precisos sin alterar ni romper el resto del sistema.



3.3.3 Prototipado Visual Multi-Pantalla con Google Stitch

Paso 1: Protocolo de Suite Multi-Pantalla en Stitch: Ingresa a <code>stitch.withgoogle.com</code>. Google Stitch genera una sola pantalla por prompt. Para erradicar el "Efecto Monovista", el aprendiz debe construir una Suite Multi-Pantalla dentro del mismo proyecto: 1. Pantalla 1 (Auth): Pegar Prompt 1 de Login/Registro. 2. Botón '+ Add Screen': En Stitch, hacer clic en '+ Add Screen' para crear la siguiente vista independiente. 3. Pantalla 2 (Dashboard): Pegar Prompt 2 (KPIs y widgets). 4. Pantalla 3 (Explorador): Clic '+ Add Screen' y pegar Prompt 3 (Tabla interactiva de citas). 5. Pantalla 4 (Detalle 360): Clic '+ Add Screen' y pegar Prompt 4 (Expediente clínico profundo). 6. Pantalla 5 (Wizard): Clic '+ Add Screen' y pegar Prompt 5 (Formulario de agendamiento por pasos). 7. Pantalla 6 (Configuración): Clic '+ Add Screen' y pegar Prompt 6 (Ajustes del perfil y clínica). 8. Consistencia Visual: Reutilizar el mismo estilo del catálogo de 40 estilos y los mismos códigos HEX en todas las vistas.	Suite de Prompts para Google Stitch (+ Add Screen) <pre>/* PROMPT 1: AUTH & ONBOARDING (SCR-01) */ Diseña pantalla de Login/Registro split-screen para "HealthPulse" (médico vs paciente), fondo Slate 900. ----- /* PROMPT 2: DASHBOARD MÉDICO (SCR-02) */ Diseña Dashboard con Sidebar activo en 'Dashboard', 4 KPIs (Citas Hoy, Pacientes), gráfico y cola en espera. ----- /* PROMPT 3: AGENDA Y EXPLORADOR DE CITAS (SCR-03) */ Sidebar activo en 'Citas', Header con breadcrumbs, tabla de turnos, filtros (presencial/telemedicina). ----- /* PROMPT 4: EXPEDIENTE CLÍNICO 360 (SCR-04) */ Header con botón '← Volver', ficha médica del paciente, tabs: Historial, Recetas activas, Exámenes de lab. ----- /* PROMPT 5: WIZARD DE NUEVA CITA (SCR-05) */ Stepper de 3 pasos (Paciente, Doctor, Horario en verde). ----- /* PROMPT 6: CONFIGURACIÓN Y HORARIOS (SCR-06) */ Sidebar activo en 'Ajustes', formulario de horarios.</pre>
Paso 2: Extracción de Tokens de Diseño de la Suite: Una vez generada la suite completa de pantallas en Google Stitch, inspeccione los componentes e identifique: • Clases utilitarias de Tailwind CSS compartidas (<code>bg-slate-900</code>, <code>backdrop-blur-md</code>, <code>rounded-2xl</code>). • Paleta cromática institucional unificada para fondos, tarjetas y acentos. • Estructura del Sidebar persistente con el indicador de ruta activa para transferirlo al Router de Google AI Studio.	Tokens Visuales Extraídos de la Suite Paleta Primaria: ■ Cian (#0891b2) ■ Esmeralda (#059669) Superficies: Fondo general <code>bg-slate-900</code>, Tarjetas translúcidas <code>bg-slate-800/80</code> con borde <code>border-slate-700/60</code>. Tipografía: Inter / Sans-serif con jerarquía clara (<code>text-sm font-medium</code> para etiquetas, <code>text-2xl font-bold</code> para KPIs).

Tip de Prototipado: En Google Stitch, puede solicitar iteraciones parciales diciendo: "Rediseña únicamente la tarjeta KPI de ingresos para incluir un gráfico sparkline de tendencia semanal manteniendo el resto intacto".



3.3.4 Modelado y Despliegue de Base de Datos en Supabase (Backend & SSOT)

Paso 1: Configuración Inicial del Proyecto Supabase: 1. Ingrese a <code>supabase.com</code> y cree un nuevo proyecto en la organización formativa. 2. Ingrese a Authentication → Providers → Email y desactive la opción: Confirm email: OFF <i>(Permite registrar usuarios de prueba y realizar login inmediato sin requerir confirmación por correo en desarrollo).</i> 3. Diríjase a SQL Editor → New Query para ejecutar el script DDL de creación de tablas, disparadores y políticas de seguridad RLS.	Trigger de Sincronización Automática de Perfiles <pre>-- 1. Función para actualizar fecha de modificación CREATE OR REPLACE FUNCTION handle_updated_at() RETURNS TRIGGER AS \$\$ BEGIN NEW.updated_at = NOW(); RETURN NEW; END; \$\$ LANGUAGE plpgsql; -- 2. Función Trigger: Sincronizar auth.users -> public.profiles CREATE OR REPLACE FUNCTION public.handle_new_user() RETURNS TRIGGER AS \$\$ BEGIN INSERT INTO public.profiles (id, full_name, email, role) VALUES (NEW.id, COALESCE(NEW.raw_user_meta_data->'full_name', 'Usuario'), NEW.email, COALESCE(NEW.raw_user_meta_data->'role', 'user')); RETURN NEW; END; \$\$ LANGUAGE plpgsql SECURITY DEFINER;</pre>
Paso 2: Tablas Relacionales y Disparador de Usuarios: Se crea la tabla <code>public.profiles</code> vinculada a <code>auth.users</code> con eliminación en cascada, y se adjunta el trigger <code>AFTER INSERT ON auth.users</code>. Posteriormente se crean las tablas del dominio (ej. <code>appointments</code>) asignando llaves UUID generadas por <code>gen_random_uuid()</code> y vinculando el propietario con <code>owner_id UUID REFERENCES public.profiles(id)</code>.	DDL de Tablas Principales <pre>-- 3. Tabla de Perfiles de Usuario CREATE TABLE public.profiles (id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE, full_name VARCHAR(150) NOT NULL, email VARCHAR(150) NOT NULL, role VARCHAR(30) DEFAULT 'patient' CHECK (role IN ('doctor', 'patient', 'admin')), created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW()); CREATE TRIGGER on_auth_user_created AFTER INSERT ON auth.users FOR EACH ROW EXECUTE FUNCTION public.handle_new_user(); -- 4. Tabla de Dominio (Citas Médicas) CREATE TABLE public.appointments (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), owner_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE, doctor_name VARCHAR(100) NOT NULL, patient_name VARCHAR(100) NOT NULL, scheduled_at TIMESTAMPTZ NOT NULL, status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'completed', 'canceled')), notes TEXT, created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW());</pre>

Regla Inviolable de Base de Datos: Toda tabla debe utilizar llaves primarias de tipo UUID con `gen_random_uuid()` para impedir ataques de enumeración secuencial en la API pública.



Paso 3: Activación Obligatoria de Row Level Security (RLS) al 100%: En Supabase, exponer una tabla sin RLS permite que cualquier persona con la anon key acceda o elimine datos ajenos. Procedimiento Inviolable: 1. Habilitar RLS en cada tabla pública mediante <code>ALTER TABLE [tabla] ENABLE ROW LEVEL SECURITY;</code> 2. Crear políticas granulares e independientes para las 4 operaciones del CRUD (<code>SELECT</code>, <code>INSERT</code>, <code>UPDATE</code>, <code>DELETE</code>). 3. Garantizar que en la inserción se asigne <code>auth.uid()</code> al <code>owner_id</code> y en la lectura/modificación se valide estrictamente <code>USING (auth.uid() = owner_id)</code>.	Script SQL de Políticas RLS Granulares <pre>-- HABILITACIÓN Estricta DE RLS ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY; ALTER TABLE public.appointments ENABLE ROW LEVEL SECURITY; -- POLÍTICAS PARA PROFILES CREATE POLICY "Permitir lectura de perfiles autenticados" ON public.profiles FOR SELECT TO authenticated USING (true); CREATE POLICY "Los usuarios solo editan su propio perfil" ON public.profiles FOR UPDATE TO authenticated USING (auth.uid() = id); -- POLÍTICAS PARA APPOINTMENTS (SSoT Y AISLAMIENTO) CREATE POLICY "Usuarios leen solo sus citas propias" ON public.appointments FOR SELECT TO authenticated USING (auth.uid() = owner_id); CREATE POLICY "Usuarios insertan citas vinculadas a su UID" ON public.appointments FOR INSERT TO authenticated WITH CHECK (auth.uid() = owner_id); CREATE POLICY "Usuarios actualizan únicamente sus citas" ON public.appointments FOR UPDATE TO authenticated USING (auth.uid() = owner_id); CREATE POLICY "Usuarios eliminan únicamente sus citas" ON public.appointments FOR DELETE TO authenticated USING (auth.uid() = owner_id);</pre>
Paso 4: Obtención de Credenciales de Conexión: Diríjase a Project Settings → API y copie exclusivamente los siguientes valores: • Project URL: La URL base de su proyecto (ej. <code>https://xyzcompany.supabase.co</code>). • Project API Keys → anon (public): Clave pública segura para clientes frontend. ALERTA DE SEGURIDAD CRÍTICA: NUNCA use ni exponga la clave <code>service_role</code> en el código de frontend. La <code>service_role</code> bypasses (ignora) todas las políticas de RLS y le otorgaría control total al usuario sobre toda la base de datos.	Inicialización Segura del Cliente Supabase <pre>// Archivo: src/lib/supabaseClient.js import { createClient } from '@supabase/supabase-js'; const supabaseUrl = import.meta.env.VITE_SUPABASE_URL 'http s://su-proyecto.supabase.co'; const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY 'su-anon-key-publica'; export const supabase = createClient(supabaseUrl, supabaseAnon Key, { auth: { persistSession: true, autoRefreshToken: true, detectSessionInUrl: true } });</pre>

Verificación Técnica: En el panel de Supabase → **Table Editor**, verifique que todas las tablas tengan el candado verde **"RLS Enabled"** antes de proceder a la generación de la interfaz en Google AI Studio.



3.3.5 Construcción y Orquestación Fullstack en Google AI Studio Apps

Paso 1: Acceso a Google AI Studio Apps: Ingresa a <code>aistudio.google.com/apps</code> con su cuenta de Google. Esta suite permite crear y desplegar aplicaciones web en tiempo real mediante Gemini 1.5 Pro / Flash. Estructura del Prompt Maestro Multi-Vista: El Prompt Maestro unifica los 4 documentos canónicos, el esquema SQL ejecutado en Supabase y la suite de pantallas de Stitch: Arquitectura Multi-Vista (SPA Router): Enrutador por estado <code>currentView</code> y Sidebar interactivo para navegar entre todas las vistas sin recargar. Componentes Desacoplados por Pantalla: Módulos independientes para Dashboard, Listado, Detalle 360, Wizard y Ajustes. Credenciales Seguras: Inyección de URL y ANON KEY en singleton. Implementación de los 4 Estados UI: Código explícito para Skeleton, Empty State, Toast y Error Alert. Manejo de Errores RLS: Captura de excepciones de base de datos (código <code>42501</code>).	Prompt Maestro Multi-Vista para AI Studio Actúa como Lead Fullstack Engineer. Construye la SPA "HealthPulse Telemed" con ARQUITECTURA MULTI-VISTA. STACK Y CONEXIÓN: - React 19 + Tailwind CSS + Lucide + @supabase/supabase-js. - Conecta a Supabase con URL y ANON_KEY. ROUTER MULTI-VISTA Y SIDEBAR ACTIVO (App.jsx): - Estado: <code>currentView</code> ('dashboard' 'citas' 'detalle' 'nueva-cita' 'ajustes') y <code>selectedId</code>. - Sidebar con enlaces activos y navegación a TODAS las vistas. - Header con breadcrumbs dinámicas según la pantalla activa. VISTAS MODULARES INDEPENDIENTES: 1. DashboardView: 4 KPIs en vivo + cola de espera. 2. AppointmentsView: Tabla interactiva + selector Kanban. 3. PatientDetailView: Expediente 360 con tabs e historial. 4. AppointmentCreateView: Wizard por pasos con validación. 5. SettingsView: Perfil médico y horarios de atención. REGLAS SSOT Y 4 ESTADOS: - 100% de datos desde Supabase. Skeleton loader en cargas, Toasts accesibles al mutar y Empty states ilustrados.
Paso 2: Implementación del Contexto de Autenticación: El archivo <code>src/context/AuthContext.jsx</code> debe proveer el estado de la sesión, el usuario autenticado y escuchar activamente los cambios de estado mediante <code>supabase.auth.onAuthStateChange</code>. De esta manera, cuando el usuario inicia sesión o cierra sesión, toda la aplicación reacciona de forma automática protegiendo las vistas privadas.	src/context/AuthContext.jsx <pre>import React, { createContext, useContext, useEffect, useState } from 'react'; import { supabase } from '../lib/supabaseClient'; const AuthContext = createContext({}); export const AuthProvider = ({ children }) => { const [user, setUser] = useState(null); const [loading, setLoading] = useState(true); useEffect(() => { supabase.auth.getSession().then(({ data: { session } }) => { setUser(session?.user ?? null); setLoading(false); }); const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, session) => { setUser(session?.user ?? null); setLoading(false); }); return () => subscription.unsubscribe(); }, []); return (<AuthContext.Provider value={{ user, loading }}> {loading && children} </AuthContext.Provider>); }; export const useAuth = () => useContext(AuthContext);</pre>



Paso 3: Capa de Servicios Modulares (appointmentService.js): Queda estrictamente prohibido realizar consultas dispersas a Supabase dentro de los componentes visuales de React. Toda comunicación con la base de datos debe encapsularse en un archivo de servicio dedicado con manejo estructurado de errores (try / catch). Observe cómo la función de inserción asigna automáticamente el owner_id del usuario activo para cumplir con la política de seguridad RLS de Supabase.	src/services/appointmentService.js <pre>import { supabase } from '../lib/supabaseClient'; export const appointmentService = { async getAll() { const { data, error } = await supabase .from('appointments') .select('*') .order('scheduled_at', { ascending: true }); if (error) throw error; return data; }, async create(appointmentData, userId) { const { data, error } = await supabase .from('appointments') .insert([{ ...appointmentData, owner_id: userId }]) .select() .single(); if (error) throw error; return data; }, async delete(appointmentId) { const { error } = await supabase .from('appointments') .delete() .eq('id', appointmentId); if (error) throw error; } };</pre>
Paso 4: Componentes con los 4 Estados de Pantalla: En el componente de vista (AppointmentList.jsx), se manejan las variables reactivas de estado: • <code>isLoading</code>: Renderiza el componente de carga <code><AppointmentSkeleton /></code>. • <code>error</code>: Renderiza la alerta de error con botón de reintento. • <code>appointments.length === 0</code>: Renderiza el <code><EmptyState /></code> con mensaje e invitación a crear. • <code>appointments.length > 0</code>: Renderiza la tabla completa con datos de Supabase.	Renderizado Condicional de Estados en React <pre>if (isLoading) { return <AppointmentTableSkeleton rows={4} />; } if (error) { return (<div className="p-4 bg-red-50 border border-red-200 rounded-lg text-red-700"> <p className="font-bold">Error al sincronizar con Supabase:</p> <p className="text-sm">{error.message}</p> <button onClick={loadAppointments} className="mt-2 px-3 py-1 bg-red-600 text-white text-xs rounded"> Reintentar conexión </button> </div>); } if (appointments.length === 0) { return <EmptyState onNewAppointment={() => setIsModalOpen(true)} />; } return <AppointmentTable data={appointments} onDelete={handleDelete} />;</pre>



3.3.6 Protocolo de Parches Quirúrgicos en Google AI Studio

Manejo de Errores sin Desbordar Contexto:

Si durante la compilación en AI Studio se presenta un error de sintaxis o de base de datos, **NUNCA** solicite: "Corrige el error" o "Reescribe la aplicación" (esto causa alucinaciones y pérdida de código funcional).

Utilice la fórmula del **Parche Quirúrgico**:

"En el archivo [NombreArchivo.jsx], ajusta ÚNICAMENTE la función [nombreFuncion] para que [comportamiento esperado]. NO toques el estado global, ni los estilos de otros componentes, ni la configuración de Supabase."

Ejemplo de Prompt Quirúrgico

```
// CASO REAL: Error 42501 (Violación de RLS en Inserción)
"En el archivo appointmentService.js, ajusta ÚNICAMENTE
la función create(). Asegúrate de inyectar explícitamente
el campo owner_id con el valor de session.user.id
antes de enviar la carga útil a Supabase:
const payload = { ...data, owner_id: user.id };
No modifiques las funciones getAll() ni delete()."
```

Resultado: La IA reescribe exactamente 5 líneas de código, preservando la integridad del 100% del proyecto restante.

3.3.7 Aseguramiento de la Calidad: Testing de los 4 Estados y CI/CD en la Nube

Testing Automatizado con Playwright y Vitest:

Para certificar que una aplicación cumple el estándar de calidad de producción, el aprendiz debe implementar pruebas automáticas para cada uno de los 4 estados de pantalla:

- **Pruebas Unitarias (Vitest):** Validan que los servicios (appointmentService.js) llamen correctamente a Supabase y procesen errores 42501 sin lanzar excepciones no controladas.
- **Pruebas E2E (Playwright):** Automatizan la navegación del usuario en Chromium headless, verificando la aparición secuencial del Skeleton Screen, el Toast verde de confirmación y el Empty State.

Test E2E de los 4 Estados (Playwright)

```
import { test, expect } from '@playwright/test';

test('Flujo completo de 4 Estados en Citas', async ({ page }) => {
  await page.goto('/dashboard');

  // 1. Verificar Skeleton mientras carga
  await expect(page.locator('.skeleton-pulse')).toBeVisible();

  // 2. Verificar Empty State inicial si no hay citas
  await expect(page.locator('text=No tienes citas agendadas')).toBeVisible();

  // 3. Crear cita y verificar Toast de Éxito
  await page.click('button:has-text("+ Agendar Cita)');
  await page.fill('input[name="doctor"]', 'Dr. Mendoza');
  await page.click('button:has-text("Confirmar")');
  await expect(page.locator('.toast-success')).toHaveText('Cita agendada');

  // 4. Verificar nueva fila persistida en la tabla
  await expect(page.locator('table')).toContainText('Dr. Mendoza');
});
```

Pipeline CI/CD con GitHub Actions y Despliegue en Vercel:

Todo cambio confirmado en la rama `main` del repositorio debe disparar un flujo automatizado de integración continua que ejecuta linters, pruebas de tipado y despliega a producción en Vercel / Netlify.

Observabilidad en Producción: Se integra **Sentry** para rastrear excepciones no controladas en tiempo real y **PostHog** para analítica de eventos de usuario, completando las 7 fases del SDLC profesional.

Workflow GitHub Actions (.github/workflows/ci.yml)

```
name: CI/CD Pipeline
on: [push, pull_request]

jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm run test:unit
      - name: Deploy to Vercel
        if: github.ref == 'refs/heads/main'
        run: npx vercel --prod --token ${ secrets.VERCEL_TOKEN }
```

Estándar de Calidad: Un desarrollo guiado por IA sólo se considera concluido cuando supera el 100% de las pruebas automatizadas y se encuentra desplegado en una URL pública accesible con persistencia en Supabase.



3.4 Actividades de Transferencia del Conocimiento:

Descripción de la actividad: Aplicación autónoma y en equipo de la metodología completa de Desarrollo Guiado por IA al Proyecto Formativo propio asignado en el programa ADSO. Ejecución del ciclo de verificación cruzada Maker-Checker entre pares técnicos.

Ambiente requerido: Ambiente de formación de desarrollo con acceso completo al ecosistema tecnológico (Gemini, Stitch, Supabase, AI Studio).

Estrategias o técnicas didácticas activas: Aprendizaje Basado en Proyectos (ABPr), evaluación cruzada por pares (Maker-Checker Protocol), sustentación técnica y auditoría en vivo.

Materiales de formación: Repositorio Git del equipo, cuentas configuradas en las plataformas, rúbrica de auditoría.

Material de apoyo: Guía de auditoría técnica SSoT y lista de verificación del Marco PIC 2026.

Duración de la actividad: 40 horas.

3.4.1 Taller Aplicado al Proyecto Formativo del Aprendiz

Cada equipo de desarrollo debe ejecutar el ciclo integral para codificar y desplegar el módulo funcional principal de su Proyecto Formativo SENA (ej. Control de Acceso de Personal, Gestión de Inventario de Almacén, Facturación Electrónica, Gestión Clínica o Flujo de Equipos Externos), entregando los 7 artefactos de la cadena técnica:

- Tetralogía Documental Aprobada:** Carpetas con los 4 documentos canónicos en formato Markdown (`01_PLAN` , `02_PRD` con escenarios Gherkin, `03_USER_FLOWS` con Catálogo de 5 a 8 Pantallas y `04_TRD`).
- Base de Datos en Producción (Supabase):** Script SQL DDL ejecutado con llaves primarias UUID, trigger `handle_updated_at` , sincronización de usuarios con `auth.users` y políticas de RLS habilitadas y comprobadas al 100%.
- Prototipo Validado en Google Stitch (Suite Multi-Pantalla):** Suite completa de pantallas (mínimo 5 a 8 vistas) generada en el mismo proyecto mediante `+ Add Screen` , con estilo unificado del catálogo de 40 estilos y los 4 estados de UI.
- Aplicación Web Funcional en Google AI Studio:** SPA React interactiva multi-vista con Sidebar navegable y enrutador por estado (`currentView`) entre todas las pantallas, conectada a Supabase y con diseño responsive.

3.4.2 Protocolo de Auditoría Cruzada Maker-Checker

N°	Punto de Verificación Técnica	Evidencia Concreta a Comprobar	Cumplimiento
01	SSoT en Base de Datos	No existen arreglos hardcoded en <code>useState</code> ni datos simulados. Todo proviene de Supabase.	[] Sí [] NO
02	Seguridad RLS al 100%	Todas las tablas tienen <code>ENABLE ROW LEVEL SECURITY</code> y políticas para SELECT, INSERT, UPDATE, DELETE.	[] Sí [] NO
03	Protección de Secretos	El frontend utiliza únicamente <code>SUPABASE_ANON_KEY</code> . La clave <code>service_role</code> NO aparece en el código.	[] Sí [] NO
04	Identificadores UUID	Todas las llaves primarias son de tipo UUID generadas mediante <code>gen_random_uuid()</code> .	[] Sí [] NO
05	Estado de Carga (Loading)	Se muestra un Skeleton Loader con <code>animate-pulse</code> mientras se resuelve la promesa de Supabase.	[] Sí [] NO
06	Estado Vacío (Empty State)	Cuando la tabla no tiene filas, muestra ilustración amigable y botón CTA para agregar el primer elemento.	[] Sí [] NO
07	Feedback de Éxito (Toast)	Tras crear, editar o borrar registros, la UI emite una notificación flotante accesible (Toast).	[] Sí [] NO
08	Manejo de Errores (Retry)	Si la consulta falla o se viola RLS (42501), la UI no colapsa en blanco; muestra banner de error y botón de reintento.	[] Sí [] NO
09	Sincronización de Perfiles	Al crear un nuevo usuario con Supabase Auth, el trigger inserta automáticamente su fila en <code>public.profiles</code> .	[] Sí [] NO
10	Límite de Modularidad	Ningún archivo de componente supera las 250 líneas de código; se dividen en subcomponentes limpios.	[] Sí [] NO
11	Navegación Multi-Vista SPA	La app posee al menos 5 vistas funcionales desacopladas (Dashboard, Lista, Detalle, Wizard, Configuración) navegables desde el Sidebar sin recargas.	[] Sí [] NO



4. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN EN EL PROCESO FORMATIVO

Fase del Proyecto Formativo	Actividad del Proyecto Formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Ejecución	AP03: CODIFICAR Y DESPLEGAR LOS MÓDULOS DEL SISTEMA DE INFORMACIÓN APLICANDO METODOLOGÍAS DE DESARROLLO GUIADO POR INTELIGENCIA ARTIFICIAL.	AA09: Construir aplicaciones web completas mediante el pipeline Gemini Gems, Google Stitch, Supabase y Google AI Studio Apps.	Evidencias de Conocimiento: Cuestionario técnico sobre la Tetralogía Documental Canónica, BDD con Gherkin, arquitectura SSOT en Supabase, políticas RLS y los 4 estados de pantalla en frontend reactivo.	• Modela los requisitos y arquitectura de software empleando estándares BDD y diagramas relacionales. • Argumenta con solvencia el impacto del Row Level Security (RLS) en la integridad y privacidad de datos.	Valoración de Conocimiento: Cuestionario estructurado en plataforma LMS SENA.
			Evidencias de Desempeño: Observación directa del proceso de configuración de la Cadena de Gemas en Gemini, prototipado visual en Google Stitch, ejecución del script DDL en Supabase y ensamblaje de la SPA en Google AI Studio.	• Aplica el protocolo de parches quirúrgicos ante errores de compilación sin degradar el contexto de la IA. • Configura el singleton de Supabase y el listener de autenticación según buenas prácticas de seguridad.	Técnica: Observación directa. Instrumento: Lista de chequeo de desempeño.
			Evidencias de Producto: Repositorio Git que incluye los 7 artefactos del pipeline, script SQL DDL con RLS verificado y enlace web a la aplicación compilada y conectada en vivo a Supabase.	• El software desarrollado persiste datos reales en Supabase sin depender de mocks en memoria. • Implementa contractualmente los 4 estados de UI (Empty, Loading Skeleton, Success Toast, Error Alert). • Cumple el 100% de los puntos de la auditoría técnica Maker-Checker.	Técnica: Valoración de producto. Instrumento: Lista de verificación / Rúbrica técnica de producto.

Evaluación por Proyectos: La aprobación de la presente guía de aprendizaje está condicionada a la sustentación técnica en vivo donde cada aprendiz demuestra el funcionamiento de la aplicación conectada a Supabase y expone la trazabilidad de los 7 artefactos generados.



5. GLOSARIO DE TÉRMINOS

- **BaaS (Backend as a Service):** Modelo en la nube para infraestructura, base de datos, autenticación y storage vía APIs unificadas (ej. Supabase).
- **BDD (Behavior-Driven Development):** Metodología ágil que define el comportamiento del sistema mediante escenarios de negocio *Dado / Cuando / Entonces*.
- **Context Rot:** Fenómeno donde el LLM pierde coherencia o alucina código debido a la saturación de su ventana de contexto.
- **Gherkin:** Lenguaje formal estructurado legible por humanos y máquinas para pruebas de aceptación (`GIVEN` , `WHEN` , `THEN`).
- **Google AI Studio Apps:** Entorno para compilar y ejecutar prototipos fullstack asistidos por Gemini con previsualización reactiva en vivo.
- **Google Stitch:** Herramienta para traducir requisitos y prompts en interfaces web de alta fidelidad con tokens de diseño.
- **JWT (JSON Web Token):** Estándar abierto (RFC 7519) para transmitir identidad y autorización cifrada y firmada digitalmente.
- **Maker-Checker:** Procedimiento donde un desarrollador construye ("Maker") y un auditor independiente ("Checker") verifica el apego a estándares.
- **Marco PIC 2026:** Metodología de desarrollo guiado por IA basada en *Precision, Isolation* y *Context*.
- **RLS (Row Level Security):** Seguridad en PostgreSQL que restringe filas accesibles según el identificador criptográfico del usuario (`auth.uid()`).
- **Skeleton Screen:** Patrón visual con animación de pulso que imita la estructura de datos mientras se resuelven promesas de red.
- **SSoT (Single Source of Truth):** Principio donde cada dato reside en una única ubicación autoritativa (PostgreSQL), prohibiendo mocks en frontend.
- **Tetralogía Canónica:** Conjunto de 4 planos previos al código: 01 Plan de Proyecto, 02 PRD, 03 User Flow y 04 TRD.
- **UUID (Universally Unique Identifier):** Identificador de 128 bits único a nivel global, utilizado como PK para evitar ataques de enumeración.

6. REFERENTES BIBLIOGRÁFICOS

Construya o cite documentos de apoyo para el desarrollo de la guía, según lo establecido en la guía de desarrollo curricular (BIBLIOGRAFÍA / WEBGRAFÍA).

- Kendall, K. E., & Kendall, J. E. (2018). *Análisis y Diseño de Sistemas* (8ª ed.). Pearson.
- Martin, R. C. (2018). *Clean Architecture: Software Structure and Design*. Prentice Hall.
- PostgreSQL Global Group. (2025). *PostgreSQL 16 Manual: Row Security Policies & Triggers*.
- Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del Software* (9ª ed.). McGraw-Hill.
- Sommerville, I. (2019). *Ingeniería del Software* (10ª ed.). Pearson Educación.
- Supabase Inc. (2026). *Supabase Documentation: Database, Auth, RLS & Realtime*.
- Google Cloud & Google AI. (2026). *AI Studio Apps & Gemini 1.5/2.0 API Developer Guide*.
- Servicio Nacional de Aprendizaje (SENA). (2024). *Modelo Pedagógico de FPI por Proyectos*.

7. CONTROL DEL DOCUMENTO

Nombre	Cargo	Dependencia	Fecha
Miguel Ángel Tejedor Mendoza	Instructor Técnico ADSO	Coordinación Académica — Centro de Gestión Agroempresarial del Oriente / SENA	Noviembre de 2024
Comité Técnico Curricular	Revisor Pedagógico	Red de Conocimiento en Informática, Diseño y Desarrollo de Software	Noviembre de 2024

8. CONTROL DE CAMBIOS (DILIGENCIAR ÚNICAMENTE SI REALIZA AJUSTES A LA GUÍA)

Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Miguel Ángel Tejedor Mendoza	Instructor	Coordinación Académica	25 de noviembre 2024	Se actualizó al formato actual GFPI-F-135 V04.
Miguel Ángel Tejedor Mendoza	Instructor	Coordinación Académica	01 de septiembre 2026	Actualización pedagógica mayor: Integración del Marco PIC 2026, Cadena de 4 Gemas Gemini, Google Stitch, Supabase RLS y Google AI Studio Apps.